

SMART PARKING SYSTEM

Task Sheet

Duration: 5 hours | Total Marks: 100

What You're Building -

Build a Smart Parking System using the provided hardware within **5 hours**. Your system should detect parking slot occupancy, control entry using RFID authentication, monitor gas/fire safety, and display live data on a web dashboard. You may also implement AI-based vehicle detection using your mobile phone camera as an advanced feature. Complete the tasks in order—**partial marks will be awarded for partially completed work**. Final evaluation will also consider how many hardware modules/sensors are successfully integrated and displayed with live data on the web dashboard.

Your Hardware Kit (same for every team) -

- 1 × ESP32-WROOM-32 board — the "brain" you'll program
- 2 × IR sensors — one for each parking slot (A, B), detects if a car is there
- 1 × MQ gas sensor — detects gas/smoke in the air
- 1 × Fire sensor — detects flame/fire risk nearby
- 1 × SG90 servo motor — acts as your gate barrier
- 1 × RFID-RC522 reader + card(s) — scans ID cards at the gate
- 1 × 0.96" OLED screen — shows status at the entrance
- Your own mobile phone camera — used for the AI tasks (no ESP32-CAM needed)

Part I- Hardware

TASK 1 — Detect if a Parking Spot is Full

Difficulty: **Easy** Marks: 10

Make each of the 2 parking slots know whether a car is there, using the IR sensors.

You'll use: 2× IR sensors, ESP32

- Connect both IR sensors to the ESP32 and read their signal
- Give each sensor a name in your code: Slot A, Slot B
- Count and show how many slots are free vs. taken overall
- Show slot statuses (Empty / Occupied) on the Serial Monitor using state-change detection and non-blocking timing (millis). Don't write a simple loop that infinitely floods the console with text.

TASK 2 — Build the Entry Gate

Difficulty: **Easy-Medium** Marks: 10

Only open the gate for cars with a valid ID card. If the lot is full, don't let anyone in — even with a valid card.

You'll use: RFID-RC522, servo motor

- Scan an RFID card and read its ID number correctly

- On a valid RFID card scan, rotate the servo from 0° to 90° to open the gate and close it again after a few seconds, automatically
- The system must not open the gate for unrecognized cards. Furthermore, if the parking lot is full, the gate must stay closed even when a valid card is scanned.
- Show the Serial Monitor all Data.

TASK 3 — Show Status on a Screen

Difficulty: Easy **Marks:** 10

Show the parking status on a small screen (like Picture 1) that a driver could read at the entrance.

You'll use: OLED display

- Connect the OLED screen and show a welcome message on power-up
- Show live status of both slots on the screen
- Show an access status message on the screen when a driver scans a card at the entrance, then return to the normal display
- Ensure smooth screen updates without visual flickering, freezing, or lag

Part II- IoT and Web

TASK 4 — Cloud-Integrated Gas and Fire Safety Alert (Firebase)

Difficulty: Medium **Marks:** 10

Send everything happening on your hardware up to the internet, so it's not stuck on just one board.

You'll use: MQ gas sensor, Fire sensor, ESP32 WiFi + Firebase

- Read gas levels from the MQ sensor, note what “normal air” looks like, and pick a level that means “danger” (test with an unlit lighter or blowing on the sensor)
- Read the fire sensor and detect a flame/fire risk nearby (test carefully with a lighter flame at a safe distance)
- Sound a buzzer or flash a light/message when either gas or fire crosses the danger level, and clear the alert automatically once things return to normal
- Create and log in to your Firebase Project and connect the ESP32 to WiFi
- Send all the live data — parking slot status, gas alerts, and fire alerts — to Firebase, organized clearly

TASK 6 — Build a Live Website (Dashboard)

Difficulty: Medium **Marks:** 10

Build a website that shows everything from Firebase, live, without needing to refresh the page.

You'll use: Firebase + any web stack

- Build a webpage that connects to Firebase and shows real data
- Show each slot as a colored card or icon (green = free, red = occupied)
- Show how many slots are free out of the total, updating live
- Show a live list of who entered: card ID, allowed/denied, and time
- Show a warning banner on the website when there's an active gas alert
- Show a live notification in the webpage when a car enters the parking lot after scanning an RFID card.

TASK 7 — Design the Dashboard and Make It Useful

Difficulty: **Medium–Hard**

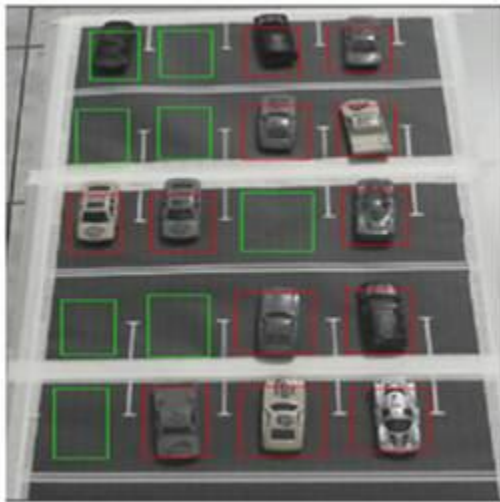
Marks: 10

Add features that make the dashboard genuinely useful for a staff member running the parking lot.

You'll use: Web stack

- Add a history page listing past entries/exits with real times
- Show a parking fee per car, based on how long it's been parked $\times$ a rate you choose
- Add a button to switch between dark mode and light mode
- Make sure the site looks and works well on both computer and phone screens, in short the web app should be responsive
- Let users search/filter the history log by date or by card/plate ID
- Your marks will reflect the effort put into the front-end design and user experience

Part III- AI



TASK 8 (Bonus) — Count Cars and Detect slots with Your Phone Camera + AI

Difficulty: **Hard** Marks: 10

Count vehicles and detect slots using your phone's camera and an AI model — no special camera module needed.

You'll use: Mobile phone camera + browser/AI object detection

- Get your phone's camera feed showing live inside the web app
- Use an AI model (e.g. TensorFlow.js/COCO-SSD, or a YOLO variant) to detect cars in the video
- Define virtual parking slots on the screen and detect when a car's bounding box stays inside one
- Cross-reference the AI visual occupancy data with the physical IR sensor data
- Push the AI slot status and entry/exit counts to your Firebase database